HUMANS.md — Working with AI on DART

This guide is for you, the human. Its companion, `AGENTS.md`, is written for the AI coding assistant. Coding agents such as Copilot, Cursor, Codex, and others can use `AGENTS.md` as the repository source of truth for DART's workflow rules. This file covers what the AI cannot do for you: setting intent, prompting clearly, and reviewing what it changed.

What the two files do

- `AGENTS.md` teaches the agent DART's architecture rules: where workflow behavior lives, what generated files to avoid, how metadata fields should be handled, and when not to make broad edits.
- `HUMANS.md` teaches you how to direct and review an agent that already knows those rules.

If a repository rule changes, update `AGENTS.md`. Treat it as authoritative so the files do not drift. `CLAUDE.md` in this repo is only a pointer back to `AGENTS.md`.

The mental model

DART is a workflow application, not just a codebase. Most requests fall into one of five buckets:

1. A workflow behavior in `app.py`
2. A mapping or default in `seeklight_mapping_template.json`
3. A helper script that repairs or normalizes metadata/config files
4. User-facing documentation for one workflow function
5. Packaging or launcher behavior

Your job with AI is to describe the workflow outcome you want, then scan the diff to make sure it touched the right bucket.

Prompting: outcomes, not file edits

The agent usually works better if you describe the workflow result you want instead of telling it which file to open.

Instead of	Try
"Edit `app.py` to change Seeklight fields"	"Make Function 5 map Seeklight Title into the standard `title` CSV field"
"Change the JSON template"	"Update the shipped Seeklight mapping defaults to match CollectionBuilder CSV field names"
"Fix the rename script"	"Make the legacy field cleanup tools remove `dc_` prefixes instead of adding them"
"Update the docs"	"Document this as a required step to keep DART in sync with collectionbuilder-csv"

Include concrete nouns when you have them: function number, field name, setting name, file type, or exact error text.

Reviewing the diff: red flags

For routine work, stop and review closely if the AI:

- Edits backup files such as `app.py.bak` or `app.py.bak2`
- Modifies old DMGs, generated CSVs, or anything in `logfiles/`
- Changes packaging scripts when the request was only about metadata or workflow behavior
- Rewrites large sections of `app.py` for a small behavior tweak
- Reintroduces `dc_`-prefixed CSV fields as the preferred convention
- Updates `README.md` but not the matching `FUNCTION_*.md` file for the same behavior
- Touches encryption, Azure settings, or identifier persistence without explaining why

A clean change usually touches one or more of these and little else:

- `seeklight_mapping_template.json`
- one focused section of `app.py`
- one helper script
- the matching function markdown doc
- `README.md` or `CHANGELOG.md` when the change is repo-wide

When to slow the agent down

Ask the AI to explain its plan before editing when the change is broad or hard to undo:

- Anything that changes object ID generation or `file_to_id_map`
- Bulk metadata rewrites across many files or folders
- Changes to Function 4 or Function 6 merge semantics
- Changes to encryption, settings persistence, or Azure path handling
- Refactors of `app.py` beyond a local fix

Going bigger

For larger work such as a new workflow function, UI redesign, or architectural cleanup, point the agent at an existing pattern and ask for a plan first.

Useful anchors in this repo:

- `FUNCTION_0_APP_SETTINGS.md` for how user-facing workflow docs are written
- `ARCHITECTURE.md` for design rationale that should not be broken casually
- `BEST_PRACTICES.md` for filename and grouping rules
- existing helper scripts for how targeted migration tools are structured

Sample prompts

A workflow behavior change:

> Update Function 6 so the merge review makes data-loss cases more obvious, but keep the current filename/objectid matching model. Show me the local plan first and update the matching function

> documentation.

A metadata normalization change:

> Remove the remaining `dc_` assumptions from the legacy cleanup scripts and docs, but do not touch user backup files or historical generated CSVs.

A settings change:

> Add a new Function 0 setting for [behavior]. Keep the existing encrypted-settings model, document it in the Function 0 help file, and explain where the value is validated before you edit.

A packaging change:

> Update the macOS packaging flow to [goal], but keep the installed runtime layout under `~/DART/`. Outline the affected scripts before changing them.

Working rhythm

1. One change at a time
2. Validate the touched slice before moving on
3. Read the matching function doc when the change affects a numbered workflow function
4. Keep `AGENTS.md` current as DART evolves so future sessions inherit better rules
5. Remember that your direct instructions override `AGENTS.md`; do not accidentally ask the AI to violate your own repository rules

Where to find documentation

Use these repo docs to ground AI requests and reviews:

- `README.md` for overall workflow and setup
- `ARCHITECTURE.md` for design decisions
- `BEST_PRACTICES.md` for filename/grouping expectations
- `FUNCTION_*.md` files for per-function behavior
- `RENAME_METADATA_FIELD.md` and `FIXING_RENAME_ISSUES.md` for metadata normalization issues
- `CHANGELOG.md` for recent behavior changes